1. Problem Title & Source

Title: **Maximum Value Thief Can Steal**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

A thief plans to steal valuables from a store.

There are **N items**, and each item has:

- **value[i]** → value of the item
- **weight[i]** → weight of the item

The thief has a bag that can carry a **maximum weight of W**.

The goal of the thief is to **maximize the total value of items stolen** without exceeding the weight limit.

Important rule:

- The thief can **take fractions of items if needed**.

This means we can take **part of an item**, not necessarily the whole item.

Your task is to determine the **maximum total value the thief can steal**.

3. Examples

Example 1

Input

3

60 10

100 20

120 30

50

Output

240

Explanation

Items:

Value	Weight
60	10
100	20
120	30

Compute value per weight:

60/10 = 6

100/20 = 5

120/30 = 4

Pick items with highest ratio first.

Step 1

Take item (60,10)

Remaining capacity = 40

Total value = 60

Step 2

Take item (100,20)

Remaining capacity = 20

Total value = 160

Step 3

Take 20/30 fraction of item (120,30)

Value gained = 80

Total

240

Example 2

Input

2

100 20

50 10

10

Output

50

Explanation

Take item with best value/weight ratio.

4. Constraints

1 ≤ N ≤ 1000

1 ≤ weight[i] ≤ 1000

1 ≤ value[i] ≤ 1000

1 ≤ W ≤ 10^4

Observations

- Fractional selection allowed
- Greedy approach works

5. Core Concept (Pattern)

This problem is the classic:

Fractional Knapsack Problem

Key technique:

Greedy + Sorting by value/weight ratio

6. Thought Process

Step 1 — Calculate Value Density

For every item compute

ratio = value / weight

This represents **value per unit weight**.

Step 2 — Sort Items

Sort items in **descending order of ratio**.

Items giving the **most value per weight** should be chosen first.

Step 3 — Pick Items Greedily

If item fits fully

take full item

If not

take fraction of item

7. Visualization

Example

W = 50

Items = [(60,10),(100,20),(120,30)]

Ratios

6, 5, 4

Order

(60,10) → (100,20) → (120,30)

Selection

10 + 20 + 20

Total value

60 + 100 + 80 = 240

8. Pseudocode

read N

for each item

```
    read value and weight

read W

compute value/weight ratio

sort items by ratio descending

totalValue = 0

for each item

    if weight ≤ W
        take full item
        W -= weight
        totalValue += value

    else
        take fraction
        totalValue += ratio × W
        break

print totalValue
```

9. Code Implementation

Python

```
n = int(input())

items = []

for _ in range(n):
    v, w = map(int,input().split())
    items.append((v,w,v/w))

W = int(input())

items.sort(key=lambda x: x[2], reverse=True)
```

```
total = 0

for v,w,r in items:

    if w <= W:
        total += v
        W -= w
    else:
        total += r * W
        break

print(int(total))
```

Java

```
import java.util.*;

class Item {

    int value, weight;
    double ratio;

    Item(int v, int w){
        value = v;
        weight = w;
        ratio = (double)v/w;
    }
}

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
```

```
        List<Item> items = new ArrayList<>();

        for(int i=0;i<n;i++){

            int v = sc.nextInt();
            int w = sc.nextInt();

            items.add(new Item(v,w));
        }

        int W = sc.nextInt();

        items.sort((a,b) -> Double.compare(b.ratio,a.ratio));

        double total = 0;

        for(Item it : items){

            if(it.weight <= W){

                total += it.value;
                W -= it.weight;
            }

            else{

                total += it.ratio * W;
                break;
            }
        }

        System.out.println((int)total);
    }
}
```

10. Time & Space Complexity

Time Complexity

Sorting items

O(N log N)

Processing items

O(N)

Total

O(N log N)

Space Complexity

O(N)

For storing items.

11. Common Mistakes

1. Treating it as **0/1 Knapsack**
2. Forgetting fractional selection
3. Sorting by value instead of ratio
4. Incorrect division calculation

12. Detailed Dry Run

Input

3

60 10

100 20

120 30

W = 50

Sorted by ratio

(60,10)

(100,20)

(120,30)

Steps

Take (60,10)

Take (100,20)

Take 20 weight from (120,30)

Value

60 + 100 + 80 = 240

13. Real-Life Applications

Fractional knapsack appears in:

- Cargo loading optimization
- Investment portfolio allocation
- Resource allocation problems
- Network bandwidth allocation

14. Common Traps

1. Sorting by value only
2. Missing fractional logic
3. Using integer division
4. Confusing with DP knapsack

15. Related Problems

Greedy problems:

LC 455 Assign Cookies

LC 435 Non-overlapping Intervals

LC 406 Queue Reconstruction

16. Alternate Approaches

Approach	Complexity	Notes
Dynamic Programming	O(NW)	unnecessary
Greedy (ratio sort)	O(N log N)	optimal

17. Why This Solution Works

Choosing items with the **highest value per weight first** ensures that every unit of capacity contributes the maximum possible value.

This greedy strategy guarantees the optimal solution for the **fractional knapsack problem**.

18. Variations / Follow-Up Questions

Possible interview variations:

1. **0/1 Knapsack** (no fractions allowed)
2. Multiple knapsacks
3. Minimize weight for given value
4. Bounded knapsack problems